Lab_sheet-01

Anish Tiwari

Section→C

Roll no.→11

Experiment No. 1

Experiment Name

Implementation of Fundamental Image Processing Operations using Python and OpenCv

Program Code and Output

Step 1: Import Required Libraries

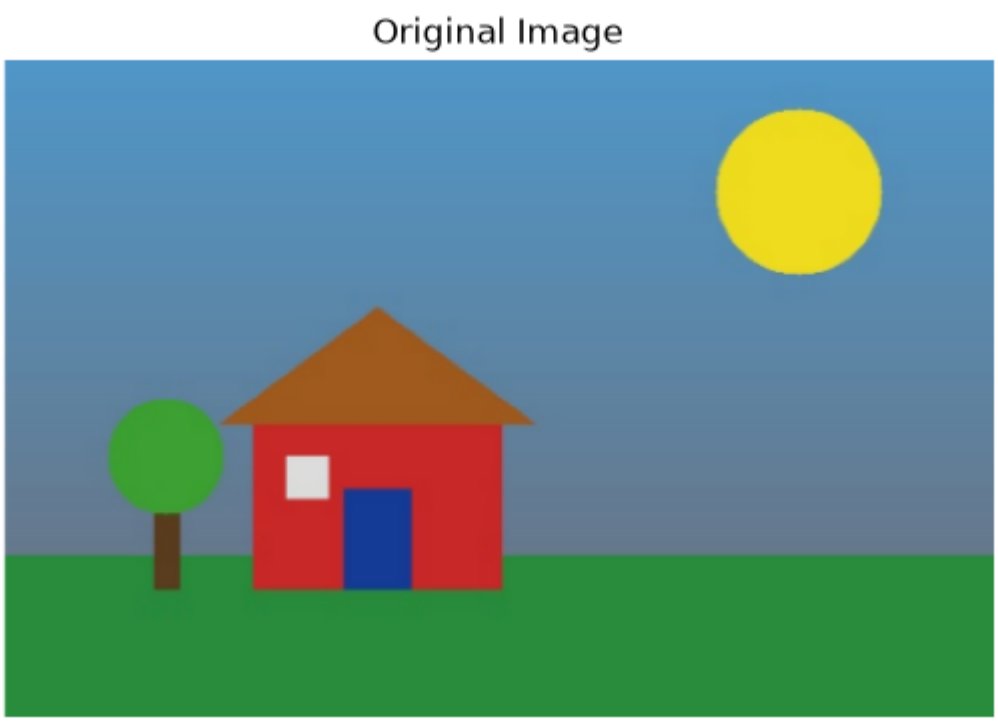
In [1]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
```

In [2]:
```
img = cv2.imread("my_pic01.png")
```

Step 2: Load and Display the Image

In [5]:
```
img_bgr = cv2.imread("my_pic01.png")            # OpenCV reads in BGR order
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)    # convert for correct display

# Display with Matplotlib
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```



Step 3: Examine Image Properties

In [6]:
```
height, width, channels = img_bgr.shape
print("Dimensions (H x W):", height, "x", width)
print("Channels:", channels)
print("Data type:", img_bgr.dtype)
print("Total pixels:", height * width)
print("Total size (bytes):", img_bgr.size)
```

```
Dimensions (H x W): 303 x 458
Channels: 3
Data type: uint8
Total pixels: 138774
Total size (bytes): 416322
```

Step 4: Save Image in Different Formats

In [7]:
```
cv2.imwrite("saved.jpg", img_bgr, [cv2.IMWRITE_JPEG_QUALITY, 90])
cv2.imwrite("saved.png", img_bgr)
```

```
print("JPEG size:", os.path.getsize("saved.jpg"), "bytes")
print("PNG size :", os.path.getsize("saved.png"), "bytes")
```

```
JPEG size: 9943 bytes
PNG size : 30225 bytes
```

Observation: The JPEG file is significantly smaller than the PNG because JPEG uses lossy compression (some pixel information is discarded), while PNG uses lossless compression (every pixel is preserved exactly, resulting in a larger file).

Step 5: Colour Space Conversion

In [8]:
```
gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
hsv  = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
lab  = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)
```

Step 6: Geometric Transformations

In [9]:
```
# Resize to 50% of original size
resized = cv2.resize(img_bgr, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)

# Rotate by 45 degrees about the centre
(h, w) = img_bgr.shape[:2]
center = (w // 2, h // 2)
M = cv2.getRotationMatrix2D(center, 45, 1.0)
rotated = cv2.warpAffine(img_bgr, M, (w, h))

# Flip
flipped_h = cv2.flip(img_bgr, 1)   # horizontal
flipped_v = cv2.flip(img_bgr, 0)   # vertical
```

Step 7: Image Complement (Negative)

In [10]:
```
negative = 255 - img_bgr          # invert every pixel intensity
cv2.imwrite("negative.png", negative)
```

Out[10]:
```
True
```

Step 8: Crop a Region of Interest (ROI)

In [11]:
```
roi = img_bgr[150:320, 90:320]     # rows 150-320, columns 90-320 (the house region)
print("ROI shape:", roi.shape)
```

```
ROI shape: (153, 230, 3)
```

Step 9: Display Original vs Processed Images

In [12]:
```
plt.figure(figsize=(14, 8))

plt.subplot(2, 4, 1)
plt.imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")

plt.subplot(2, 4, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(2, 4, 3)
plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.title("HSV")
plt.axis("off")

plt.subplot(2, 4, 4)
plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title("Rotated 45°")
plt.axis("off")

plt.subplot(2, 4, 5)
plt.imshow(cv2.cvtColor(flipped_h, cv2.COLOR_BGR2RGB))
plt.title("Flipped (H)")
plt.axis("off")

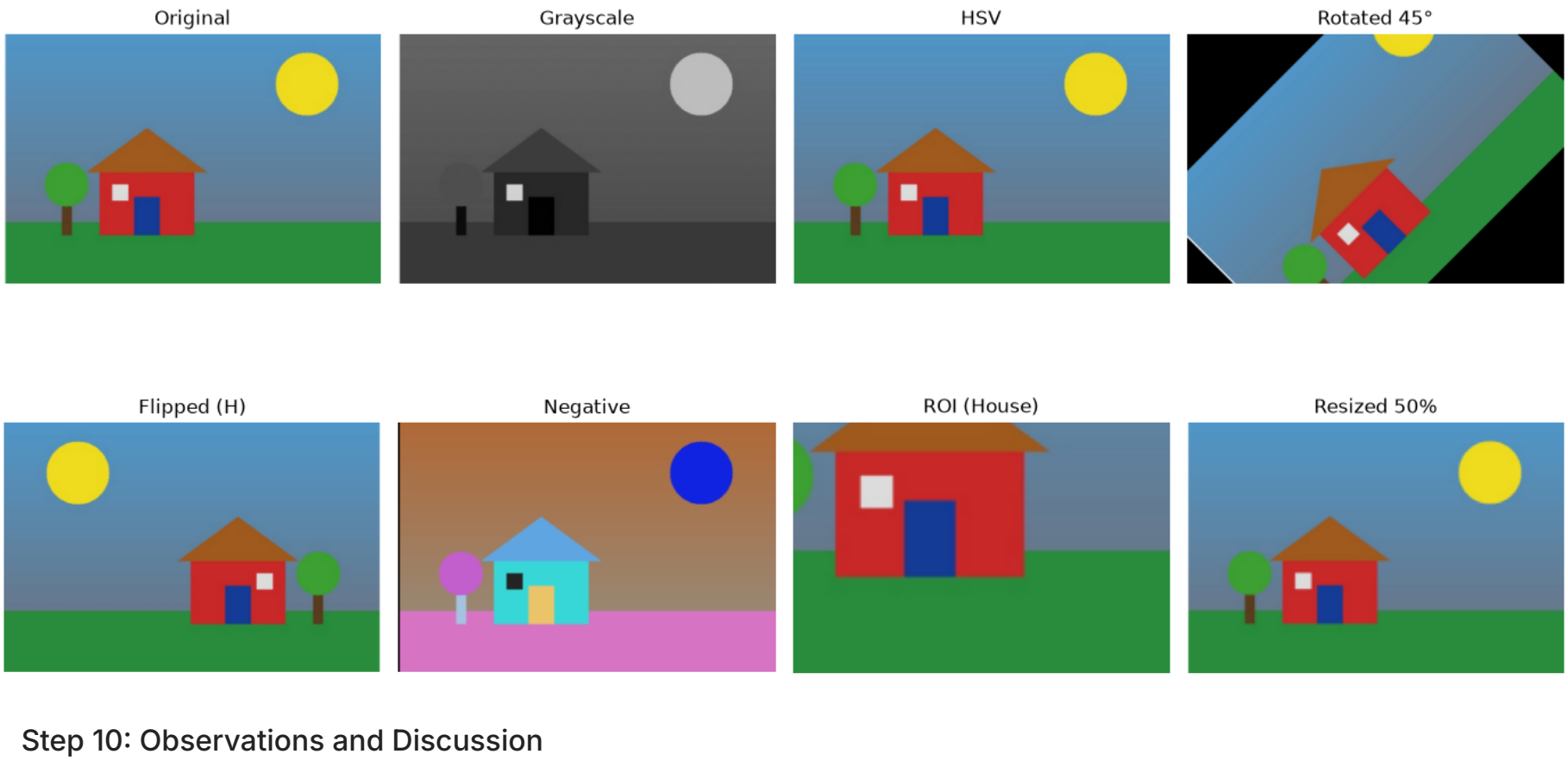
plt.subplot(2, 4, 6)
plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.title("Negative")
plt.axis("off")

plt.subplot(2, 4, 7)
plt.imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
plt.title("ROI (House)")
plt.axis("off")

plt.subplot(2, 4, 8)
```

```
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title("Resized 50%")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Step 10: Observations and Discussion

The experiment demonstrates that a digital colour image is stored as a 3-D NumPy array (height × width × channels) with 8-bit unsigned integer values (0–255) per channel. Grayscale conversion collapses the three colour channels into a single intensity channel, reducing data volume and computational cost for further processing. HSV and LAB representations separate colour information from intensity, which makes operations such as colour-based segmentation and lighting-invariant analysis considerably easier than working directly in RGB/BGR. Geometric transformations (resizing, rotation, flipping) alter the spatial arrangement of pixels without changing pixel values, and are essential for data augmentation in deep learning pipelines. The negative transform simply inverts intensity values and is useful for highlighting details in dark regions. Cropping an ROI isolates the part of the image relevant to a task, reducing the search space for subsequent detection or recognition. Together, these preprocessing operations form the foundation on which higher-level computer vision tasks (feature extraction, object detection, classification) are built.

Viva / Theory Questions

Q1. What is a digital image? Differentiate between grayscale and color images.

A digital image is a two-dimensional representation of a visual scene, stored as a finite grid (matrix) of picture elements called pixels, where each pixel holds one or more numeric values representing brightness and/or colour. It is essentially a function f(x, y) sampled and quantised so that a computer can store and process it as an array of numbers.

A grayscale image has a single channel per pixel, representing only intensity (brightness) on a scale typically from 0 (black) to 255 (white); it carries no colour information. A colour image has multiple channels per pixel — commonly three (Red, Green, Blue in RGB, or Blue, Green, Red in OpenCV's BGR) — so each pixel is a combination of channel intensities that together produce a specific hue and brightness. Colour images therefore require roughly three times the storage of an equivalent grayscale image and carry richer visual information, while grayscale images are simpler and computationally cheaper to process.

Q2. Explain the difference between RGB, BGR, HSV, and LAB color spaces.

RGB (Red, Green, Blue) represents a pixel as an additive mixture of red, green and blue intensities and is the standard colour space used by displays, image files, and most image-processing libraries such as Matplotlib and PIL.

BGR is identical to RGB in concept but stores the same three channels in reverse order (Blue, Green, Red). OpenCV uses BGR as its default channel order for historical reasons, so images loaded with cv2.imread() must be converted to RGB before being displayed correctly with Matplotlib.

HSV (Hue, Saturation, Value) separates colour information (Hue — the actual colour, Saturation — colour purity/intensity) from brightness (Value). This separation makes HSV far more convenient than RGB/BGR for tasks like colour-based object detection or segmentation, since a specific colour can be isolated by thresholding the Hue channel alone regardless of lighting changes.

LAB (or CIELAB) represents colour using L (Lightness), A (green–red component) and B (blue–yellow component). It is designed to be perceptually uniform, meaning numerical differences between LAB values correspond closely to perceived colour differences by the human eye, which makes it useful for colour correction, comparison, and consistent colour-based processing across different lighting conditions.

Q3. What is the purpose of converting an image to grayscale before further processing?

Converting to grayscale reduces a three-channel image to a single intensity channel, which (a) reduces memory usage and computational cost by roughly two-thirds, (b) simplifies algorithms such as edge detection, thresholding and feature extraction that depend primarily on intensity gradients rather than colour, and (c) removes colour variation as a confounding factor when the shape, texture or structure of objects — not their colour — is what matters for the task. Many classical computer vision algorithms (e.g., Canny edge detection, Haar cascades) are designed to operate on grayscale input for exactly these reasons.

Q4. Explain the concept of image complement (negative image) and mention its practical applications.

The complement (negative) of an image is obtained by inverting every pixel intensity value, typically computed as (255 – pixel value) for an 8-bit image. Bright regions become dark and dark regions become bright, while the structural content of the image is preserved. Practical applications include: enhancing white or light details embedded in predominantly dark regions of medical images such as X-rays and mammograms; producing photographic-negative effects; improving the visibility of features in astronomical or low-light photography; and serving as a preprocessing step before other enhancement operations such as histogram-based methods.

Q5. Differentiate between image resizing, cropping, and scaling.

Resizing changes the pixel dimensions (height and width) of the entire image, typically using interpolation (e.g., nearest-neighbour, bilinear, bicubic) to estimate new pixel values — the full scene is retained but rendered at a different resolution. Cropping extracts a rectangular sub-region of the original image without altering the resolution of the retained pixels — it discards everything outside the selected region rather than resampling the whole image. Scaling is closely related to resizing and specifically refers to changing the image size by a multiplicative factor (e.g., scale factor of 0.5 to halve dimensions, or 2.0 to double them) while preserving the aspect ratio; it is essentially a proportional form of resizing.

Q6. What is a Region of Interest (ROI), and why is it important in computer vision?

A Region of Interest (ROI) is a specific sub-area of an image — usually defined by pixel coordinates — that contains the object or feature relevant to a particular task, isolated from the rest of the image. In OpenCV/NumPy, an ROI is extracted simply by slicing the image array, e.g., img[y1:y2, x1:x2]. ROIs are important because restricting processing to the relevant region reduces computational cost, eliminates irrelevant background noise that could confuse detection or classification algorithms, and allows task-specific operations (e.g., face recognition on a detected face region, license-plate reading on a detected plate region) to be applied only where needed rather than across the entire image.

Q7. Why is OpenCV preferred over conventional image processing libraries for computer vision applications?

OpenCV is preferred because it is a mature, highly optimised C/C++ library (with Python bindings) offering real-time performance, which is critical for video and camera-based applications. It provides an extensive, ready-to-use set of functions covering the full computer vision pipeline — image I/O, filtering, geometric transformations, feature detection, object tracking, camera calibration, and integration with deep-learning frameworks (DNN module). It supports hardware acceleration (multi-threading, GPU/CUDA), works across platforms (Windows, Linux, macOS, mobile, embedded devices), and has a large, active open-source community and extensive documentation, making it more scalable and production-ready than general-purpose image libraries such as PIL/Pillow, which are simpler but lack the breadth of algorithms and real-time performance needed for advanced computer vision work.

Q8. Explain how image resolution and pixel intensity influence image quality.

Resolution refers to the number of pixels used to represent an image (height × width). Higher resolution means more pixels are available to capture fine detail, producing sharper, more detailed images, but at the cost of larger file size and higher computational/storage requirements. Lower resolution images appear blocky or blurred because fine details are lost when multiple real-world details are averaged into a single pixel. Pixel intensity refers to the brightness value stored at each pixel (e.g., 0–255 for 8-bit images). The bit-depth determines how many distinct intensity levels can be represented — higher bit-depth allows smoother gradients and finer distinctions between similar shades, reducing visible banding, while low bit-depth causes visible steps between intensity levels (posterisation). Together, resolution (spatial quality) and intensity depth (tonal quality) determine the overall visual fidelity of a digital image and directly affect how well downstream algorithms such as edge detection or feature extraction can perform.

Q9. Mention five real-world applications where basic image preprocessing is an essential step.

• Medical imaging — enhancing and normalising X-rays, MRI, and CT scans before diagnosis or tumour-detection algorithms are applied.

• Autonomous driving — converting and resizing camera feed frames, correcting colour/lighting variations before lane and object detection.

• Surveillance and security — background subtraction, denoising and contrast enhancement of CCTV footage for motion or facial recognition.

• Optical Character Recognition (OCR) — grayscale conversion, thresholding and noise removal on scanned documents before text extraction.

• Satellite and remote-sensing imagery — colour-space conversion and geometric correction before land-cover classification or crop-health monitoring.

Q10. How do image preprocessing techniques improve the performance of feature extraction and deep learning models?

Preprocessing improves downstream model performance in several ways. Normalisation and resizing produce fixed, consistent input dimensions and pixel-value ranges that neural networks require, and prevent features with larger raw values from disproportionately influencing training. Noise removal and contrast/colour correction remove irrelevant variation, helping the model focus on genuinely discriminative patterns rather than sensor noise or lighting artefacts, which reduces overfitting and improves generalisation. Grayscale conversion or colour-space transformation can reduce dimensionality where colour is not discriminative, speeding up training and inference. Data-augmentation operations built from geometric transformations (rotation, flipping, cropping, scaling) artificially expand the training dataset and make models more robust to variations in object position, orientation and scale that occur in real-world deployment. Overall, well-preprocessed input leads to faster convergence, higher accuracy, and better generalisation of both classical feature-extraction algorithms (e.g., SIFT, HOG) and deep learning models (e.g., CNNs).